

BRIC-DT

Building Reliable Interfaces

for Composable Digital Twins

Gomes, Wright, Larsen, Esterle, Zech, Cavalcanti, Woodcock, **Oakes**, Vangheluwe

EDTconf 2026 · MODELS Companion · Málaga · Tuesday 6 October 2026





Authors



Cláudio Gomes

 Aarhus University



Thomas Wright

 Aarhus



Sir Peter Gorm Larsen

 Aarhus



Lukas Esterle

 Aarhus



Philipp Zech

 Innsbruck



Ana Cavalcanti

 York



Jim Woodcock

 Southwest
 York



Bentley Oakes

 Montréal



Hans Vangheluwe

 Antwerp



Name



BRICDT



brick·ed *adjective*

of a device, **irreversibly broken**

“the firmware update bricked my router”



Name



BRIC-DT

Building **R**eliable **I**nterfaces for **C**omposable **D**igital **T**wins



Idea: *compose digital twins like Lego bricks*



Is it this easy to snap DTs together?

Structure ✓ checks exist

- datatypes
- port directions
- mirrored connections

Behaviour ✗ rarely checked

- validity
- fidelity
- stability
- timing
- robustness
- uncertainty

Vision: contract-aware DT interfaces
that state behavioural assumptions and guarantees

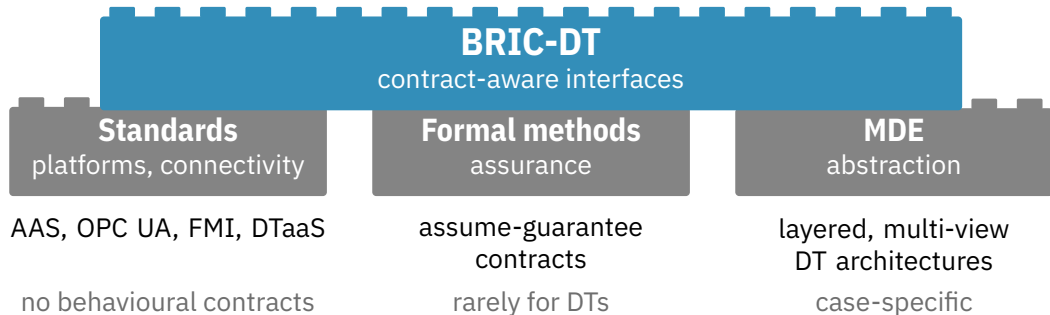
Muctadir et al. (2025) A Consistency Management Framework for DT Models

Wright et al. (2022) Formally Verified Self-adaptation of an Incubator DT

Oakes et al. (2023) Examining Model Qualities and Their Impact on DTs



Building on Existing Techniques



Khedr & Fitzgerald (2025) Composition of DTs for Systems-of-Systems: A Systematic Literature Review

Zhang et al. (2025) DT and the Asset Administration Shell

Talasila et al. (2024) Composable DTs on DT as a Service Platform

Zech et al. (2026) A Generalized Feature Model for DTs



Assume-Guarantee Contracts



Assume: what each component needs

Guarantee: what it then promises



A: inputs within validity frame

G: output error $\leq \delta_m$



A: model error $\leq \delta_m$

G: output error $\leq \delta_m + O(\Delta t^4)$

satisfies

Contracts compose when guarantees satisfy assumptions



Recurring DT Components



Model

captures system dynamics

Simulator

runs models forward in time

State estimator

combines sensor data with predictions

System ID

updates model from data

Model coupling

combines models or simulations

Optimizer

guides decisions

Knowledge manager

tracks parameters, history and errors



not exhaustive



Behavioural Qualities



fidelity

outputs stay close to real system

stability

bounded inputs give bounded outputs

timing

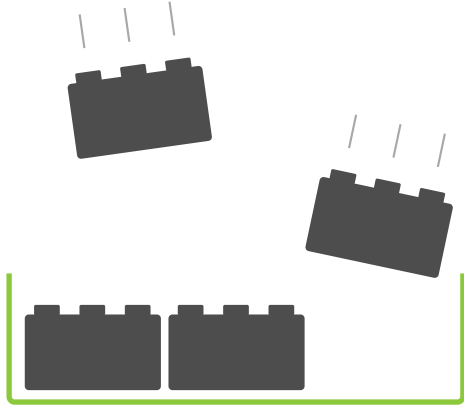
results arrive before they are needed

robustness

small input noise stays small

validity, uncertainty, relevance, verifiability, substitutability, faithfulness

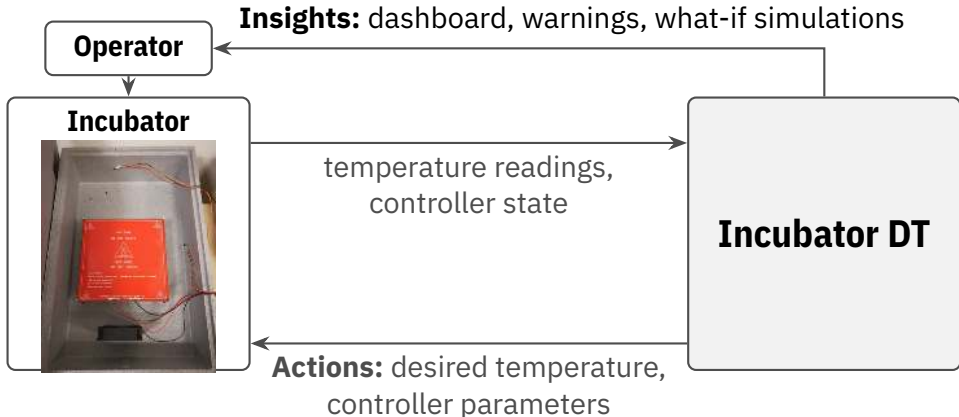
How can we state contracts,
on these components, for these qualities?



Incubator Example



Case Study: Incubator DT

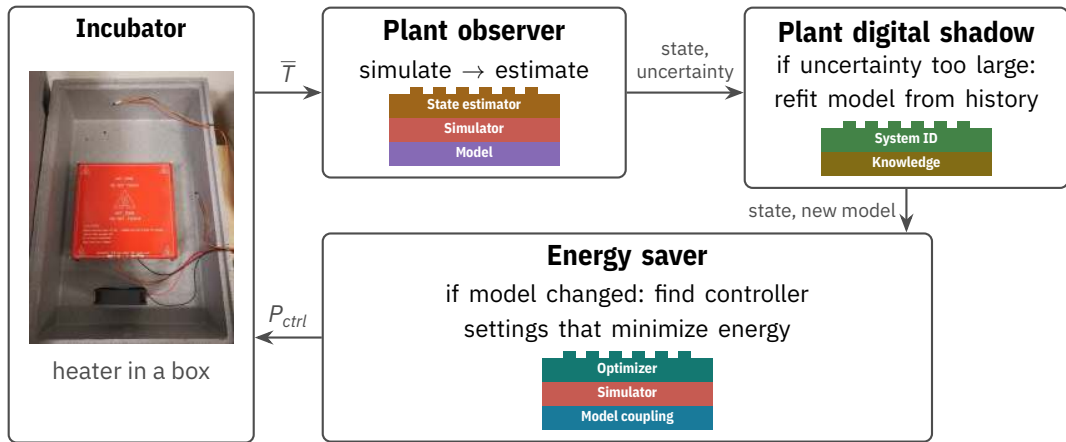


Incubator goal: maintain temperature at 37.5 °C

DT goal: save energy when plant changes (mass added, lid opened)



Incubator DT Architecture



Vision: build DT services from recurring components



Incubator DT Contracts

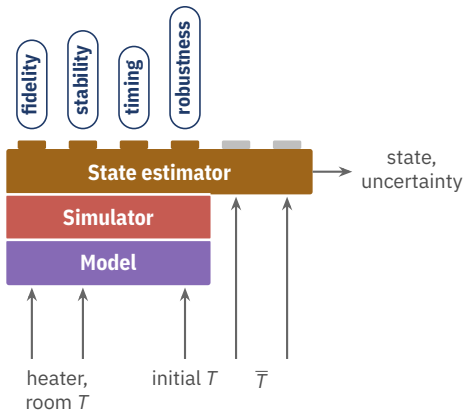


	Fidelity	Stability	Timing	Robustness
Model	A: inputs in validity frame G: residual $\leq \delta_m$	A: bounded inputs G: bounded outputs, energy conserved	A: $\emptyset$ G: derivatives in $O(n)$	A: initial cond. within δ_u G: final noise $\delta_x < \delta_u$
Simulator	A: (Model G) G: residual $\delta_m + O(\Delta t^4)$	A: (Model G) G: $\leq O(\Delta t^4)$ energy added	A: (Model G) G: cost $O(\frac{T}{\Delta t} n^3)$	A: (Model G) G: noise $< \delta_u + O(\Delta t^3)$
State Estimator	A: (Simulator G) G: residual $\delta_m + O(\Delta t^3)$	A: observable, Gaussian noise G: bounded steps	A: measurements every Δt G: done within Δt	A: residual $< \delta_m + O(\Delta t^4)$ G: residual $< f(\delta_m, \Delta t, Q, R)$
System ID	A: persistently exciting data G: residual $\leq \delta_m$	A: BIBO stable model G: local optimum	A: (Model, Simulator G) G: cost $O(Np^2 \frac{T}{\Delta t} n^3)$	A: white noise, cov. R G: sensitivity $O(1/\kappa(X))$
Optimizer	A: (Model G) G: within validity frame	A: convex cost, feasible set G: global optimum	A: (System ID G) G: cost $O(\mathcal{T} N p^2 \frac{T}{\Delta t} n^3)$	A: feasible set bounded G: sensitivity bounded
Knowledge Mgr	A: jitter within δ_t G: interpolation within δ_v	A: bounded read/write rates G: no data loss	A: bounded read/write rates G: query latency bounded	n/a

9 of 23 compose directly with assume-guarantee reasoning
What about the others?



Composing the Plant Observer

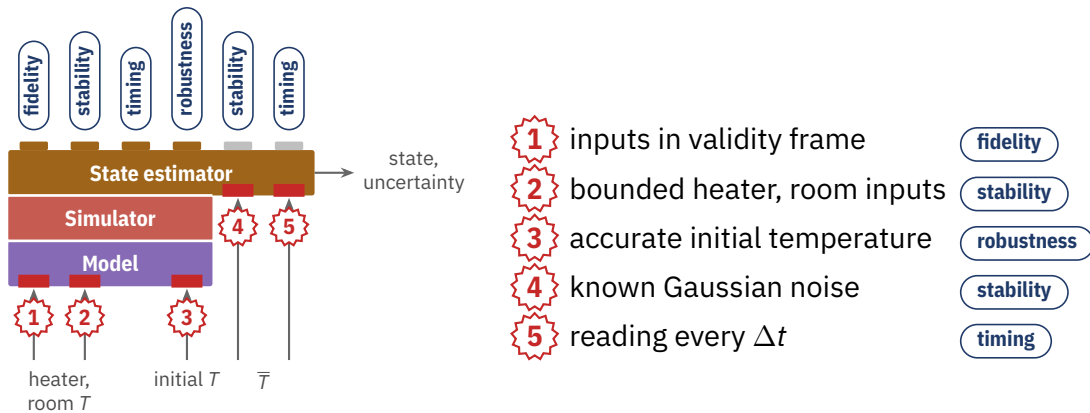


- ✓ Simulator's assumptions met by model
- ✓ Estimator's fidelity and robustness assumptions met by simulator
- ✓ Plant observer service then provides estimator's guarantees

Service contract follows from component contracts



Missing Assumptions



Plant observer service needs these five assumptions met



Checking Assumptions



**When to check,
and how?**

2 bounded heater, room inputs

 **Design** compatibility checks

3 accurate initial temperature

 **Deployment**

calibration data,
validation tests

4 known Gaussian noise

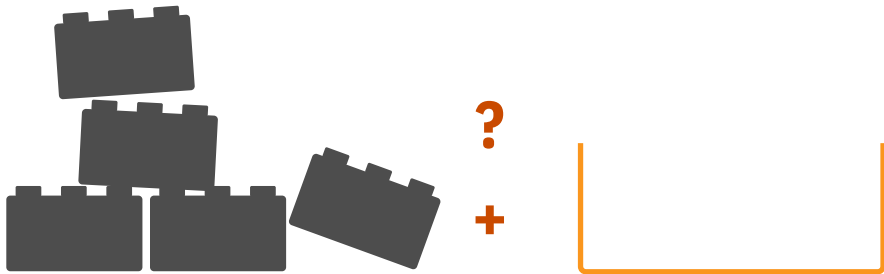
1 inputs in validity frame

 **Runtime**

monitors for latency,
operating envelope

5 reading every Δt

Vision: tools to gather evidence at each stage



Vision Directions



Direction 1: Specifying Twins



DSL for contracts?

```
component StateEstimator
assume      simulator.error <= dm + 0(dt^4)
assume      temperature every dt
guarantee estimate.error <= dm + 0(dt^3)
```

Generate checks for each stage?

 **Design** consistency, compatibility
 **Deployment** checks, tests
 **Runtime** monitors

Specify for each quality?

fidelity **stability** **timing** **robustness**
(relevance) (verifiability)
(substitutability) (faithfulness)
+ role-specific: optimizer constraints

How expressive?

- Signal temporal logic (STL)
- Specification patterns



Direction 2: Composing Twins



Which operators?

- Reusable composition operators
- Parallel, feedback, refinement, additive extension, integrative merging

Guarantees for system-level behaviour?

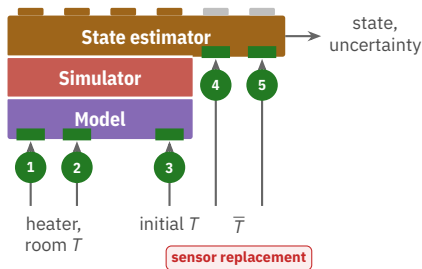
- Contracts are per component, users need system-level guarantees
- Incubator: safe operating bounds, energy targets, decision latency

How to integrate into languages?

- Reuse workflow, co-simulation languages
- Links must carry contracts, evidence
- Emergent failures: conflicting assumptions, feedback, uncertainty growth
- Monitor composed twin, not only components



Direction 3: Evolving Twins



What breaks after sensor swap?

- ✓ All five assumptions checked
- ✗ Sensor replacement reopens 4 5
- ✓ Contracts point to 4 5 to recheck
- ✓ If 4 5 hold, **digital shadow** and **energy saver** keep their evidence

Only affected assumptions need rechecking after change
assume-guarantee reasoning, abstraction, runtime monitoring



Domain engineers

- Describe operating assumptions
- State validity claims
- Use structured patterns



Formal-methods tools

- Discharge proof obligations
- Generate tests/monitors

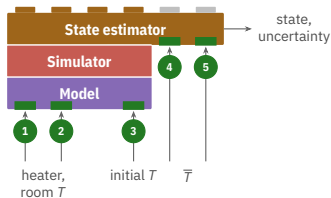


DT platforms

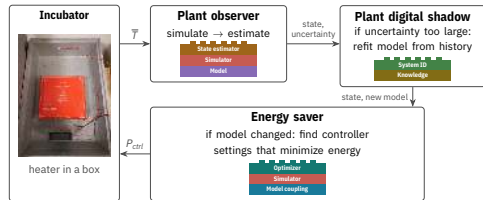
- Orchestrate components
- Check contracts



Conclusion and Takeaways



Composing DT components



Application to incubator DT

- **Behaviour** is harder to compose than structure
- Assume-guarantee **contracts per DT component**, composed across services
- Evidence at **design, deployment, and runtime**, rechecked on reuse and evolution
- Agenda: **specifying, composing** and **evolving** twins

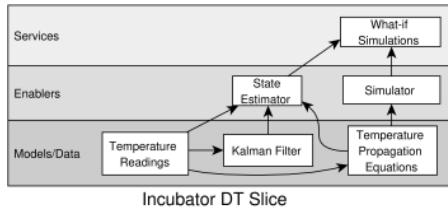
BRIC-DT: Building Reliable Interfaces for Composable Digital Twins

Gomes, Wright, Larsen, Esterle, Zech, Cavalcanti, Woodcock, **Oakes**, Vangheluwe

bentley.oakes@polymtl.ca · doi.org/10.1145/3837062.3839083



Examining Model Qualities



Q Relevancy: Does DT service's context match the system's context?

Verifiability: Can we check property satisfaction on service?

Substitutability: Does satisfaction result match the system result?

Fidelity: How closely do the results match?

Future work: Integrate into DT framework(s), deepen formalization

Examining Model Qualities and Their Impact on Digital Twins

Oakes, Gomes, Denil, Deantoni, Cambeiro, Fitzgerald, and Larsen



Towards a Component Repository



With contracts

Simulator

data schema ✓

region: validity frame

timing: $O(\frac{T}{\Delta t}n^3)$

fidelity: $\delta_m + O(\Delta t^4)$

uncertainty: $\delta_u + O(\Delta t^3)$

evidence: calibration data

State estimator

data schema ✓

region: validity frame

timing: within Δt

fidelity: $\delta_m + O(\Delta t^3)$

uncertainty: $f(\delta_m, \Delta t, Q, R)$

evidence: validation tests

Optimizer

data schema ✓

region: search region

timing: per-epoch bound

fidelity: validity frame

uncertainty: bounded

evidence: benchmarks

Store contracts and evidence with each component